

DEEP LEARNING-BASED PARTICLE IDENTIFICATION IN THE GLUOX
EXPERIMENT

A Thesis

Presented to

The Faculty of the College of Graduate Studies

Lamar University

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Computer Science

by

Anupam Siwakoti

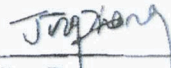
May 2025

DEEP LEARNING-BASED PARTICLE IDENTIFICATION IN THE GLUEX

EXPERIMENT

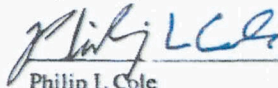
ANUPAM SIWAKOTI

Approved:



Jing Zhang

Supervising Professor



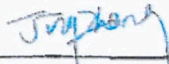
Philip L. Cole

Committee Member



Igal Jang

Committee Member



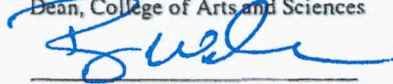
Jing Zhang

Chair, Department of Computer Science



Rebecca Ard Boone

Dean, College of Arts and Sciences



Brett Welch

Dean, College of Graduate Studies

© 2025 by Anupam Siwakoti

No part of this work may be reproduced without permission except as indicated by the “Fair Use” clause of the copyright law. Passages, images, or ideas taken from this work must be properly credited in any written or published materials.

ABSTRACT

DEEP LEARNING-BASED PARTICLE IDENTIFICATION IN THE GLUEX

EXPERIMENT

by

ANUPAM SIWAKOTI

This thesis explores the application of deep learning techniques for particle identification in the GlueX experiment at Jefferson Laboratory. The primary objective was to enhance classification accuracy using a Multi-Layer Perceptron (MLP) model trained on simulated data, generated through Monte Carlo methods. The dataset includes charged particles within a 1° to 120° polar angle range. To achieve this, a custom deep learning framework was developed using PyTorch, incorporating ReLU activation, Kaiming Initialization, and the Adam optimizer for stable and efficient training. The model was trained using mini-batches of 2048 samples with structured data preprocessing and feature selection. Performance evaluation was conducted using Precision, Recall, and F1-Score, with gradient monitoring to ensure optimization stability.

Experiments were conducted on a high-performance computing server (ifarm2402.jlab.org) equipped with a 256-core AMD EPYC 9554 processor and 1.5 TiB of RAM. The results demonstrate a significant improvement in particle classification accuracy compared to traditional methods, showcasing the potential of deep learning in high-energy physics research.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Dr. Philip L. Cole for providing me with the noble opportunity to work on this project at Jefferson Laboratory. His unwavering support, from facilitating my research involvement to arranging my stay and accommodations at Jefferson Lab, has been invaluable. His mentorship and encouragement have played a crucial role in shaping this research endeavor. I extend my deepest appreciation to Dr. Jing Zhang for his dedicated supervision throughout the project. His continuous guidance, insightful discussions, and unwavering support during both the research phase and the writing of this thesis have been instrumental in its successful completion. His expertise and patience have greatly contributed to my understanding and execution of this work. I would also like to express my heartfelt thanks to Dr. Igal Jaegle, who has been an exceptional project lead. His willingness to collaborate, address complex physics-related challenges, and provide constructive feedback has significantly enriched my learning experience. His thorough review of this thesis, particularly from the physics perspective, has been invaluable in ensuring its accuracy and depth. Finally, I am grateful to Jefferson Laboratory and all those who have contributed, directly or indirectly, to the successful completion of this project. Their support and encouragement have been essential throughout this journey.

Tables of Contents

List of Tables	vi
List of Figures	vii
Abbreviations	viii
Chapter	Page
1 Introduction	1
1.1 Background	1
1.2 Limitations of Current Methods	2
1.3 Proposed Methods	3
1.4 Thesis Objectives	5
1.5 Thesis Structure	6
2 Literature Review	7
2.1 Overview	7
2.2 Related Works	7
3 Methodology	13
3.1 Workflow Initialization and Configuration	14
3.2 Data Loading and Preprocessing	15
3.3 Model Architecture and Training Process	19
3.3.1 Architecture and Layer Configurations	19
3.3.2 Training Process	21

3.4 Post-Training Analysis and Visualization	23
4 Experiments	27
4.1 Dataset	27
4.2 Evaluation Metrics	28
4.3 Experiment Settings	30
4.3.1 Model Configuration Details	31
4.3.2 HPO Configuration Details	31
4.4 Experimental Results	34
4.4.1 Validation Performance for Positive Particles	36
4.4.2 Testing Performance for Positive Particles	38
4.4.3 Insights and Implications	38
4.5 Discussion	40
4.6 Limitations	41
5 Conclusion	43
References	44

List of Tables

Table		Page
Table 4.1	Classification Report Highlighting Precision, Recall, and F1-Score of Positive Particle Validation Dataset	35
Table 4.2	Classification Report Highlighting Precision, Recall, and F1-Score of Positive Particle Test Dataset	37

List of Figures

Figure	Page
Figure 3.1 ML flow for the classification project under GlueX setup	13
Figure 3.2 Distribution of Feature Ranges Prior to Normalization	16
Figure 3.3 Distribution of Feature Ranges Post Normalization	17
Figure 3.4 Correlation Heatmap for Selected Features Highlighting Multicollinearity	18
Figure 3.5 Correlation Heatmap for Selected Features After Mitigating Multicollinearity	19
Figure 3.6 Architecture of the Neural Network Model with ReLU and Softmax Activations	23
Figure 3.7 Gradient Flow Monitoring Across Model Layers During Training	25
Figure 4.1 Training and Validation Loss and Accuracy Trends for the Neural Net Model Over 2000 Epochs	34
Figure 4.2 Confusion Matrix for Validation Dataset Using Positive Combined-All- Charged-Particles-All-Tkin-26.0 Degree	36
Figure 4.3 Confusion Matrix for Testing Dataset Using Positive Combined-All- Charged-Partciles-All-Tkin-26.0 Degree	37
Figure 4.4 Confusion Matrix for Training and Validation with pi+ Particle Class After Applying Data Cuts	39
Figure 4.5 Confusion Matrix for Random Forest Model Using the Sample Dataset Settings as Neural Networks	41

Abbreviations

HPO	Hyper Parameter Optimization
QCD	Quantum Chromodynamics
ML	Machine Learning
HCC	Hand Crafted Cuts
DNN	Deep Neural Networks
PID	Particle Identification
CFG	Configuration

Chapter I

Introduction

1.1 Background

The study of quantum chromodynamics (QCD) forms a cornerstone in our understanding of nuclear physics. QCD, the fundamental theory describing the strong interactions among quarks and gluons, underpins the behavior of hadrons—the building blocks of atomic nuclei. These interactions are governed by the principles of confinement and asymptotic freedom, making QCD one of the most complex and intriguing components of the Standard Model of particle physics. Despite decades of research, many aspects of QCD, such as the mechanism of confinement and the emergence of hadronic spectra, remain open questions.

The GlueX experiment at Jefferson Laboratory (JLAB) in Hall D represents a significant effort to unravel the mysteries of QCD by probing the interactions that give rise to the observed spectrum of hadronic matter. By utilizing a high-intensity photon beam, the experiment aims to explore exotic states of matter, including hybrid mesons, which are hypothesized to provide critical insights into the confinement mechanism of QCD. Hybrid mesons, characterized by their unique quark-gluon structures, are believed to extend our understanding of how gluons contribute to the overall properties of hadrons.

Central to the GlueX experiment is the Bremsstrahlung technique, which generates a photon beam with energies ranging from 6 to 11.7 GeV and a resolution of 50 MeV. This precise photon tagging facilitates a detailed exploration of particle interactions and allows for the isolation of specific reaction channels critical to the study of exotic states. The GlueX detector system is engineered to track both charged particles, such as

electrons (e^-/e^+), muons (μ^+/μ^-), pions (π^+/π^-), and kaons (K^+/K^-), as well as neutral particles, including photons and neutrons. The detector's wide angular coverage, spanning polar angles from 1° to 120° , ensures comprehensive data acquisition, essential for reconstructing complex particle interactions.

Particle identification in such high-energy experiments is both critical and challenging. Accurate classification of particles and interactions not only enhances the quality of the experimental data but also aids in validating theoretical models of QCD. Traditional methods of particle identification rely on a combination of detector measurements, such as energy deposits, timing information, and trajectory reconstruction, and physics-based algorithms. These methods, while effective, often require extensive manual tuning and are susceptible to biases inherent in predefined selection criteria. Additionally, the increasing energy and intensity of modern experiments, coupled with the sheer volume of data generated, necessitate more sophisticated analytical approaches.

1.2 Limitations Of Current Methods

Despite their utility, conventional particle identification techniques face several limitations. Firstly, these methods are heavily reliant on manual calibration and domain-specific expertise, making them labor-intensive and prone to human error. The rigidity of predefined selection criteria can result in the exclusion of valid events or the misclassification of particles, thereby reducing the overall accuracy and reliability of the analysis. Additionally, the increasing complexity and volume of data generated by modern experiments pose significant challenges for traditional approaches, which may struggle to scale effectively.

Another limitation is the potential for biases introduced during the development of physics-based algorithms. These biases can skew the analysis, particularly when dealing with rare or unexpected events that deviate from established models. Furthermore, traditional methods often fail to leverage the full richness of the data, as they typically focus on specific features or metrics rather than capturing the multidimensional correlations present in the dataset.

In recent years, advancements in machine learning (ML) have revolutionized data analysis across various scientific disciplines. ML techniques offer the potential to automate and optimize particle identification by learning patterns and correlations directly from experimental data. By leveraging the GlueX dataset, this thesis explores the application of ML algorithms to particle classification, with the goal of achieving high accuracy and efficiency in identifying and categorizing particle events. This approach not only mitigates the limitations of traditional methods but also paves the way for uncovering subtle features and interactions that might otherwise remain elusive.

This thesis focuses on pre-processing the GlueX dataset to extract meaningful features, training ML models tailored for particle classification, and optimizing these models to achieve robust performance. The outcomes are expected to contribute to a deeper understanding of the fundamental physics driving particle interactions and enhance the analytical capabilities of high-energy nuclear physics experiments.

1.3 Proposed Methods

The proposed methodology leverages advanced machine learning (ML) techniques to enhance particle classification in the GlueX experiment. The first step involves extensive preprocessing of the GlueX dataset, where raw detector data is

cleaned, normalized, and transformed into meaningful observables such as momentum and energy distributions. Relevant interactions are isolated by applying event selection criteria, while background noise and irrelevant data are minimized. Next, feature extraction techniques focus on deriving key kinematic and spatial variables from the detector data, and dimensionality reduction methods, such as Principal Component Analysis (PCA) (Jolliffe, 2002), are applied to optimize the feature space while retaining critical information. A range of ML algorithms, including Random Forest, Boosted Decision Trees, and Neural Networks, are trained and validated using appropriate subsets of the dataset.

Hyperparameter tuning and cross-validation are employed to ensure the robustness and generalizability of the models. Furthermore, extensive hyperparameter optimization was performed to identify suitable parameters tailored to the particle data, enhancing classification accuracy. The methodology also includes feature engineering, such as examining feature correlation and applying feature normalization to improve model performance. Advanced optimization techniques, such as grid search and Bayesian optimization, are utilized to refine model performance. Finally, interpretability is prioritized by ensuring the models' predictions align with physical principles and experimental observations, with additional validation conducted using simulated or alternative experimental datasets. This comprehensive approach aims to establish a scalable and reliable framework for particle identification, addressing the limitations of traditional techniques and advancing the analytical capabilities of high-energy nuclear physics research.

1.4 Thesis Objectives

The primary objective of this thesis is to develop and optimize a robust Particle Identification (PID) classification model for the GlueX experiment, utilizing both Hand Crafted Cuts (HCC) and Machine Learning (ML) techniques. By integrating these models into the Jefferson Laboratory Machine Learning workflow, the study aims to enhance the identification accuracy of particles produced in high-energy nuclear physics experiments. This work will focus on improving particle classification by addressing critical challenges in current methods, such as combinatorial background and cross-contamination from incorrect identification. The PID model will be designed to identify tracks prior to reconstructing the final state, significantly improving the signal-to-background ratio. Key tasks include:

1. **Model Development:** Design and train deep neural network (DNN) models alongside HCC techniques, ensuring the models capture intricate patterns in the GlueX dataset.
2. **Feature Engineering and Hyperparameter Optimization:** Implement feature engineering techniques such as feature correlation analysis and normalization, along with extensive hyperparameter optimization, to tailor the model to particle-specific datasets.
3. **Performance Evaluation:** Evaluate the models in terms of PID purity, reproducibility of results, and ease of integration into the HallD C++ software used in GlueX.

4. Deployment and Analysis: Deploy the models for physics analysis within the GlueX experiment, facilitating enhanced discovery and ensuring the models align with reproducible and systematic evaluation standards.

Through these objectives, the thesis aims to advance particle classification methodologies, bridging gaps between traditional techniques and modern ML-based approaches, and ultimately improving the analytical capabilities of the GlueX experiment.

1.5 Thesis Structure

This thesis is structured to systematically address the development and optimization of a Particle Identification (PID) classification model. Chapter 1 introduces the topic, highlighting the background, limitations of existing methods, objectives, and the overall layout of the thesis. Chapter 2 reviews existing literature, covering traditional and advanced machine learning methods for particle identification, along with their comparative performances. Chapter 3 outlines the methodology, presenting the framework and detailed steps for data preprocessing, feature engineering, and model optimization. Chapter 4 focuses on experiments, describing the dataset, evaluation metrics, experimental settings, and results. Chapter 5 provides a comprehensive discussion on the findings, comparing proposed methods with existing approaches, and examining their implications. Finally, Chapter 6 concludes the thesis, summarizing the contributions, identifying limitations, and proposing future research directions. References are included to support the research and provide additional context.

Chapter II

Literature Review

2.1 Overview

In this chapter, I will explore existing research related to particle identification (PID) in high-energy nuclear physics, focusing specifically on studies that have made significant contributions to this field. Hundreds of papers have been published, reflecting the breadth and depth of approaches adopted to tackle the challenges of PID. I will analyze these works with particular emphasis on advancements achieved through the integration of machine learning (ML) technologies. Special attention will be given to PID techniques within the context of quantum chromodynamics (QCD) and those employing ML for performance enhancement, including the utilization of different hyperparameter optimization (HPO) techniques highlighted in various studies. This review aims to identify gaps in current methodologies, position our research within the broader scientific context, and derive insights that can inform the development of an optimized PID system for the GlueX experiment. By building upon the existing body of knowledge, this thesis endeavors to bridge identified gaps and advance the state of the art in PID techniques.

2.2 Related Works

Palo and Molzon (2024) proposed a neural network (NN)-based framework to enhance the spatial resolution of drift chambers by addressing nonlinearities and systematic distortions in drift time data. The authors trained a fully connected feedforward neural network using simulated and experimental datasets, where input features included calibrated drift times, wire positions, and environmental variables such as gas density and electric field gradients. The network was optimized using stochastic

gradient descent with regularization techniques to prevent overfitting and ensure generalization to unseen data. Advanced feature scaling and normalization methods were employed to preprocess the input data, allowing the model to learn subtle patterns and nonlinear relationships that traditional polynomial fitting methods could not capture. The resulting NN model demonstrated a significant reduction in position measurement uncertainties, achieving sub-millimeter accuracy across a range of drift distances. This improvement directly impacts track reconstruction algorithms, enhancing momentum resolution and particle trajectory determination. The study highlights the effectiveness of machine learning in mitigating hardware-induced artifacts, offering a robust, adaptable solution for modern particle tracking systems in high-energy physics experiments like GlueX.

Kai Zhou et al. (2023) demonstrated the transformative role of machine learning (ML) in studying quantum chromodynamics (QCD) under extreme conditions. The authors employed various ML techniques, including deep neural networks (DNNs) for classification, convolutional neural networks (CNNs) for feature extraction, and generative models such as variational autoencoders (VAEs) and generative adversarial networks (GANs) to reconstruct missing data and model complex distributions. Additionally, reinforcement learning (RL) was utilized to optimize simulation parameters and explore critical regions in the QCD phase diagram. Through these methodologies, the study successfully identified QCD phase transitions, such as the shift between quark-gluon plasma and hadronic matter, and pinpointed critical points with high accuracy. Advanced hyperparameter optimization techniques further enhanced model performance, ensuring the robustness and reliability of results. Their findings highlighted the potential

of ML to uncover subtle correlations and mitigate noise in experimental data, aligning with theoretical predictions. This work serves as a compelling example of how ML can advance the analysis of complex QCD phenomena, providing valuable insights for the development of ML-based particle identification systems, such as those being designed for the GlueX experiment.

Radovic et al. (2018) present a comprehensive exploration of how advanced machine learning (ML) techniques are transforming particle physics at the high-energy and high-intensity frontiers. The authors detail the application of supervised learning algorithms, such as boosted decision trees (BDTs) and deep neural networks (DNNs), for tasks like particle identification (PID) and event classification, where these models leverage the vast, high-dimensional feature spaces provided by modern detector systems. Generative models, including variational autoencoders (VAEs) and generative adversarial networks (GANs), are utilized to simulate rare physical processes and reconstruct incomplete or noisy data, thereby enhancing the fidelity of experimental analysis. Unsupervised learning techniques are applied to anomaly detection, identifying rare events that may signify new physics beyond the Standard Model.

A key technical contribution of the paper is the discussion of ML-based trigger systems for real-time data filtering, where ML models optimize the selection of high-priority events in data streams, significantly reducing storage and computational overhead without sacrificing essential information. The authors also highlight the use of hyperparameter optimization (HPO) frameworks, such as Bayesian optimization, to fine-tune model architectures for improved performance and efficiency. Furthermore, the

study emphasizes interpretability techniques, ensuring that ML models' decisions are aligned with established physical theories and experimental constraints.

Yan He et al. (2024) present a convolutional neural network (CNN)-driven framework designed to automate the detection of double- Λ hypernuclear events within nuclear emulsion data. The methodology begins with image preprocessing to enhance signal quality and reduce noise, followed using CNNs for feature extraction to identify key topological patterns, such as vertex configurations and track orientations indicative of double- Λ events. Hyperparameter optimization via grid search refined the network's parameters, improving detection accuracy, while ensemble techniques further enhanced prediction reliability. The model achieved a high detection efficiency, successfully identifying over 90% of double- Λ events in a test dataset, with significant improvements in processing speed compared to manual analysis. Additionally, the system demonstrated a robust capability to localize decay vertices, providing detailed spatial information critical for hyper nuclear event reconstruction. These results underscore the potential of ML-based automation to address challenges in nuclear emulsion analysis, offering scalable and efficient solutions relevant to particle identification tasks in high-energy experiments like GlueX.

Xiao-Yun Wang et al. (2024) applied machine learning (ML) techniques, including support vector machines (SVMs) and neural networks, to determine the strong coupling constant, a critical parameter in quantum chromodynamics (QCD). By utilizing high-dimensional input features such as momentum transfer, event shape observables, and jet substructure variables, the authors employed preprocessing methods like normalization and principal component analysis (PCA) to enhance model efficiency and

reduce dimensionality. The ML models demonstrated high accuracy in extracting the running of the strong coupling constant across energy scales, with results aligning closely with theoretical and experimental benchmarks. Hyperparameter optimization via grid search ensured robust and generalizable predictions. This work highlights the capacity of ML to tackle non-linear and high-dimensional challenges in QCD, offering a scalable approach to parameter estimation that can be extended to particle identification and event analysis in experiments like GlueX, where precise calibration of fundamental constants is crucial.

Yang et al. (2023) present a convolutional neural network (CNN)-based classifier designed to retrace the initial orientations of deformed nucleus-nucleus collisions. Utilizing data generated through the isospin-dependent Boltzmann-Uehling-Uhlenbeck (BUU) transport model for ultra-central uranium-uranium collisions at an energy of 1 GeV/nucleon, the authors develop a machine learning framework to categorize collision events based on their orientation biases. The CNN processes multiple experimental observables, integrating spatial and momentum-related features to determine the orientation of colliding nuclei. This data-driven polarization scheme filters specific orientation-biased events, enabling precise reconstruction of initial collision geometries. The benefits of this approach are substantial, as it significantly improves the resolution of deformed nucleus studies, facilitating investigations into critical nuclear properties such as symmetry energy and neutron skin thickness.

While previous research has made remarkable contributions to machine learning applications in particle physics, my thesis takes a more targeted approach by optimizing particle identification (PID) for the GlueX experiment. Many of these studies, such as

Radovic et al.'s work on machine learning at the energy and intensity frontiers of particle physics, demonstrate the power of supervised learning techniques like boosted decision trees (BDTs) and deep neural networks (DNNs) for PID and event classification. Their use of hyperparameter optimization (HPO) frameworks, including Bayesian optimization, to fine-tune model architectures has been particularly influential in shaping my approach. Similarly, Zhou et al.'s research on QCD phase transitions using deep learning and generative models inspired my exploration of feature extraction techniques to capture complex particle interactions. Building on these insights, I have not only employed HPO methods such as Bayesian optimization to enhance model efficiency but also incorporated a cut-based analysis—benchmarking feature values to refine model performance systematically. Furthermore, I have developed a streamlined pipeline that allows direct comparison between different models, such as DNN, Random Forest, and Boosted Decision Trees.

Chapter III

Methodology

In this study, a systematic and robust methodology was developed to address the challenges of particle identification in experimental physics, leveraging the PyTorch framework to construct a comprehensive data processing and model training pipeline. The raw experimental data, stored in ROOT format, was transformed into Pandas DataFrames to facilitate preprocessing and analysis. Key preprocessing steps, such as handling missing values, normalization, and feature engineering, were seamlessly integrated into the pipeline, ensuring optimal data quality for model training. This pipeline supported the implementation and training of neural networks, decision trees, and boosted decision trees, offering a versatile approach to particle classification. By adhering to principles of software engineering, including Domain-Driven Design (DDD), the methodology emphasized modularity, reusability, and scalability, enabling its application to GlueX experiment data and beyond. The workflow not only streamlined model development but also facilitated domain-specific evaluations, ensuring accuracy, robustness, and practical applicability in particle classification tasks. Figure 3.1 illustrates the flow of the project, from data processing to model creation and result visualization.

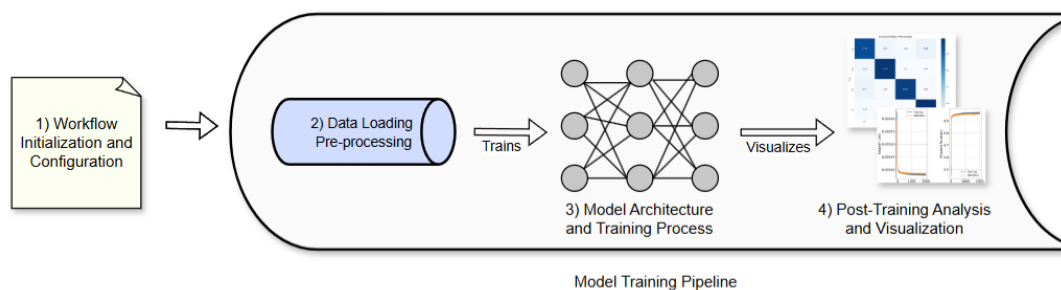


Figure 3.1: ML flow for the classification project under GlueX setup

3.1 Workflow Initialization and Configuration

The workflow begins with the Driver, which acts as the primary control mechanism for initializing and managing the model training pipeline. The Driver interacts with the CFG (Configuration) module to load critical configurations, including dataset paths, feature mappings, and hyperparameters for model training. The CFG is highly flexible, allowing users to define input features, label encodings, dropout rates, learning rates, and other essential parameters. This modular design ensures that the pipeline can easily adapt to different datasets and experimental setups.

Once the configurations are established, the model training pipeline is invoked to integrate and execute the various components. It begins by processing raw data through a parser, transforming it into a format suitable for model training. The pipeline then builds and trains the model based on the specified architecture, computes metrics for performance evaluation, and generates visualizations to track progress. By managing the interplay between data, models, and evaluations, the pipeline ensures a streamlined, efficient, and reproducible workflow for particle identification, making it a critical part of the system's design.

The model training pipeline initializes by loading and validating configuration settings for all components. These configurations, defined in JSON or Python dictionaries, specify key parameters such as data paths, feature names, model architecture, optimizer settings, and training epochs. To enhance flexibility, the pipeline supports user-defined overrides through command-line arguments, allowing researchers to adapt the workflow to various experimental conditions.

3.2 Data Loading and Preprocessing

A custom data parser was implemented to streamline the ingestion, preprocessing, and transformation of structured experimental data. The parser supports multiple file formats, including csv, JSON, feather, and root, the latter being handled via the uproot library to ensure compatibility with GlueX datasets. Configurable parameters were managed through a configuration file reader utility, enabling flexibility in specifying features, target names, data paths, random seeds, and validation split ratios. Key preprocessing steps were as follows:

1. **Dataset Splitting:** Balanced training-validation splits were performed using `train_test_split` from `sklearn`, with configurable validation ratios to ensure reproducibility.
2. **Normalization:** Features were scaled using appropriate techniques to ensure consistent ranges, improving stability and efficiency during model training.
3. **Label Encoding:** A method was applied to convert classification labels into a suitable format, allowing the model to effectively distinguish between multiple categories.
4. **Weight Initialization:** Each sample was assigned a weight initialized through a uniform random distribution, normalized to sum to one, supporting weighted loss calculations.

3.2.1 Feature Engineering

Normalization was applied to rescale the data within a standard range, specifically using Min-Max Scaling to bring all numerical values between 0 and 1. This step ensured that features with varying scales did not disproportionately influence the results of

subsequent analyses or models. Visualizations were generated to highlight the transformation of data ranges before and after normalization, showcasing the impact of preprocessing on data consistency and uniformity. These steps were crucial in preparing the dataset for further exploratory analysis and machine learning tasks. Figures 3.2 and Figure 3.3 illustrate how we rescale the data before training the model.

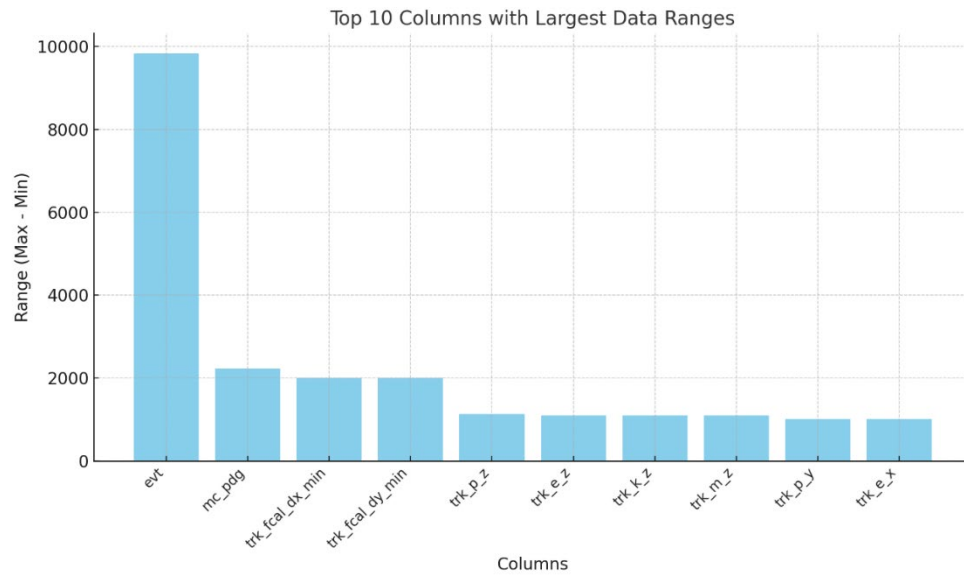


Figure 3.2: Distribution of Feature Ranges Prior to Normalization

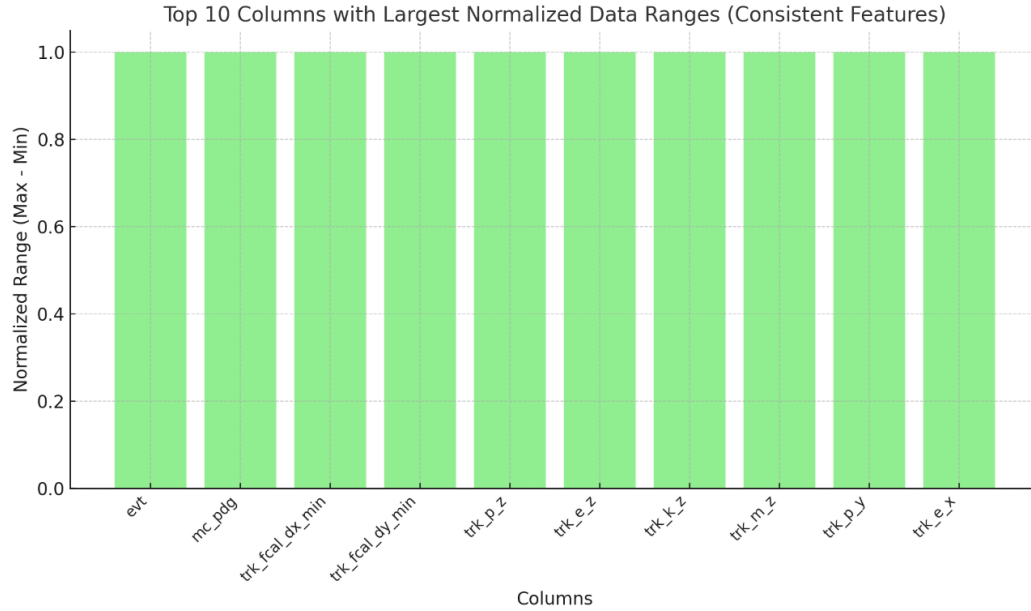


Figure 3.3: Distribution of Feature Ranges Post Normalization

To improve the dataset's quality and optimize model performance, a feature correlation analysis was performed. This analysis focused on identifying and mitigating multicollinearity, where certain features exhibit high correlations with each other, leading to redundancy and potential inefficiencies in machine learning models.

The correlation heatmaps presented in Figure 3.4 (Before Feature Optimization) and Figure 3.5 (After Feature Optimization) illustrate the results of this process. In the initial heatmap (Figure 3.4), several features, including `trk_fcal_dx_min`, `trk_fcal_dy_min`, `trk_fcal_e`, `trk_fcal_t`, and `trk_Nblk`, were found to be highly correlated with others. These features were deemed redundant as they did not provide unique information. After removing these features, the resulting heatmap (Figure 3.5) demonstrates a cleaner correlation structure, indicating reduced multicollinearity among the remaining features. This iterative refinement ensured that the dataset retained only the most relevant and non-redundant information, leading to improved model interpretability

and efficiency. The visualizations provided a clear insight into the effectiveness of the feature selection process.

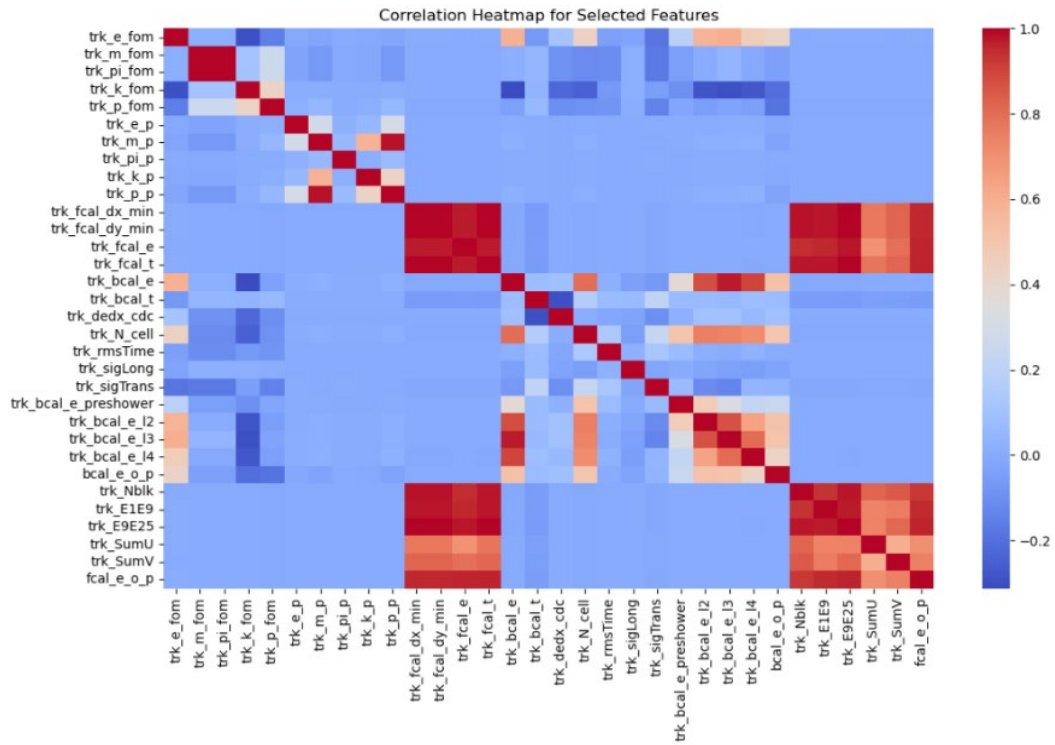


Figure 3.4: Correlation Heatmap for Selected Features Highlighting Multicollinearity

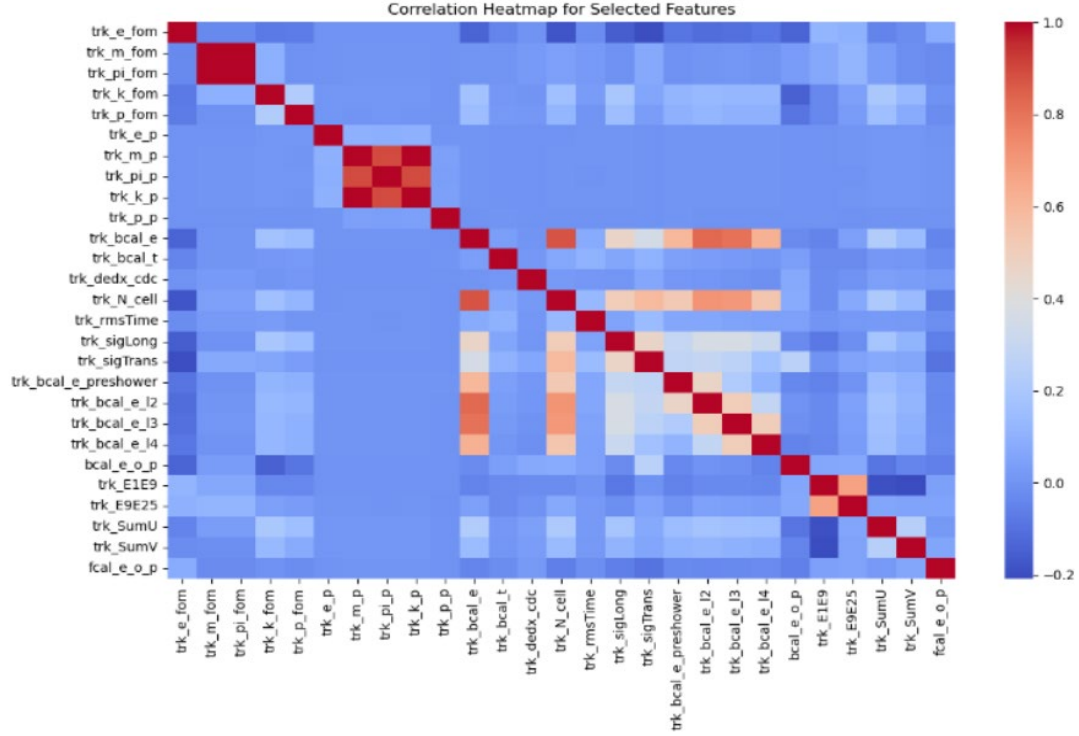


Figure 3.5: Correlation Heatmap for Selected Features After Mitigating Multicollinearity

3.3 Model Architecture and Training Process

Now that the raw ROOT data has been preprocessed into tensors, allowing seamless integration into the model training pipeline, the next step involves constructing deep learning models tailored for particle identification. To facilitate the construction of these models, a custom framework was developed to configure PyTorch functionalities. This framework enabled the dynamic construction of Multi-Layer Perceptrons (MLPs) through modular functions that utilized PyTorch's OrderedDict for the sequential integration of layers.

3.3.1 Architecture and Layer Configurations

The architecture is defined by a sequence of fully connected layers, where each layer i is represented as $h_i = f(W_i h_{i-1} + b_i)$. Here, W_i denotes the weight matrix, b_i

represents the bias vector, and f is the activation function applied element-wise. The framework supports activation functions such as ReLU, LeakyReLU, ELU, Tanh, and Sigmoid, dynamically selectable based on task-specific requirements. Based on the requirements, ReLU as the activation function was chosen. The output layer includes a Softmax activation function for multiclass classification:

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)} \quad (1)$$

where z_i represents the raw logits for class i , and C is the total number of classes.

Since the task involves classifying multiple particle types, categorical cross-entropy loss function was applied, defined as:

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{i,j} \log \hat{y}_{(i,j)} \quad (2)$$

where:

- N represents the total number of samples.
- C denotes the number of particle classes.
- $y_{i,j}$ is the true label for sample i and class j , encoded as a one-hot vector, where $y_{i,j}=1$ if sample i belongs to class j , and $y_{i,j}=0$ otherwise.
- $\hat{y}_{i,j}$ is the predicted probability for sample i belonging to class j , obtained from the model's softmax output.

This loss function measures the divergence between the true distribution (ground truth) and the predicted probability distribution for each sample. The logarithmic term $\log(\hat{y}_{i,j})$ penalizes incorrect predictions, with a heavier penalty for lower confidence in the correct class.

Kaiming Initialization for ReLU-based activations, $w \sim N\left(0, \sqrt{\frac{2}{n_i}}\right)$ Here, n_i is the number of input neurons to the layer. This scaling ensures that the variance of the activations remains consistent across layers, preventing the gradients from becoming too small or too large during backpropagation. Given that this project involves deep neural networks with ReLU activations for particle classification, Kaiming Initialization is particularly well-suited to maintain gradient flow and ensure efficient convergence.

3.3.2 Training Process

The Adam optimizer was chosen for its ability to handle noisy gradients and adaptively adjust learning rates for individual parameters, making it particularly effective for the high-dimensional datasets encountered in particle classification tasks. Adam combines the momentum of past gradients with adaptive learning rates, ensuring stable convergence and efficient optimization. Once the model is initialized, the preprocessed data, consisting of input features and corresponding labels, is fed into the neural network in mini batches as specified in the configuration. For each batch, the features (x) are passed through the network to compute predictions ($\hat{y}$), which are then compared to the true labels (y) using the categorical cross-entropy loss function. Gradients of the loss are computed through backpropagation, and the optimizer updates the model parameters to minimize the loss.

Adam calculates updates for each parameter using exponentially decaying averages of past gradients (m_t) and squared gradients (v_t)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta_t) \quad , \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(\theta_t))^2 \quad (3)$$

where β_1 and β_2 are momentum terms (default values 0.9 and 0.999, respectively).

These terms help smooth out fluctuations in gradient updates and prevent abrupt changes during optimization. The bias-corrected estimates for m_t and v_t are given by:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (4)$$

Bias correction ensures that the moving averages are unbiased during the early stages of training. The final parameter updates are computed as:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \quad (5)$$

Here, η is the learning rate, and ϵ is a small constant (default 10^{-8}) added for numerical stability. This formula adaptively scales the learning rate for each parameter based on the gradient magnitude, allowing Adam to perform well on sparse gradients and noisy datasets.

The training leveraged the Adam optimizer with a learning rate of 1×10^{-3} , chosen for its ability to handle noisy gradients and adaptively adjust learning rates for individual parameters. The model was trained over 10,000 epochs, with periodic evaluations every 5 epochs to monitor performance and gradient flow. A batch size of 2048 was used, balancing computational efficiency and gradient stability during mini-batch training. The architecture, previously described, consisted of six fully connected layers with dimensions [140, 20, 200, 110, 150, 50], using ReLU as the activation function in all hidden layers and SoftMax in the output layer for multiclass classification. Figure 3.6 illustrates the architecture of the neural network used for training. Regularization was applied through dropout layers with a rate of 0.1 across all layers to prevent overfitting. To ensure gradient stability, the training process included gradient monitoring, allowing for early detection and mitigation of issues such as vanishing or exploding gradients.

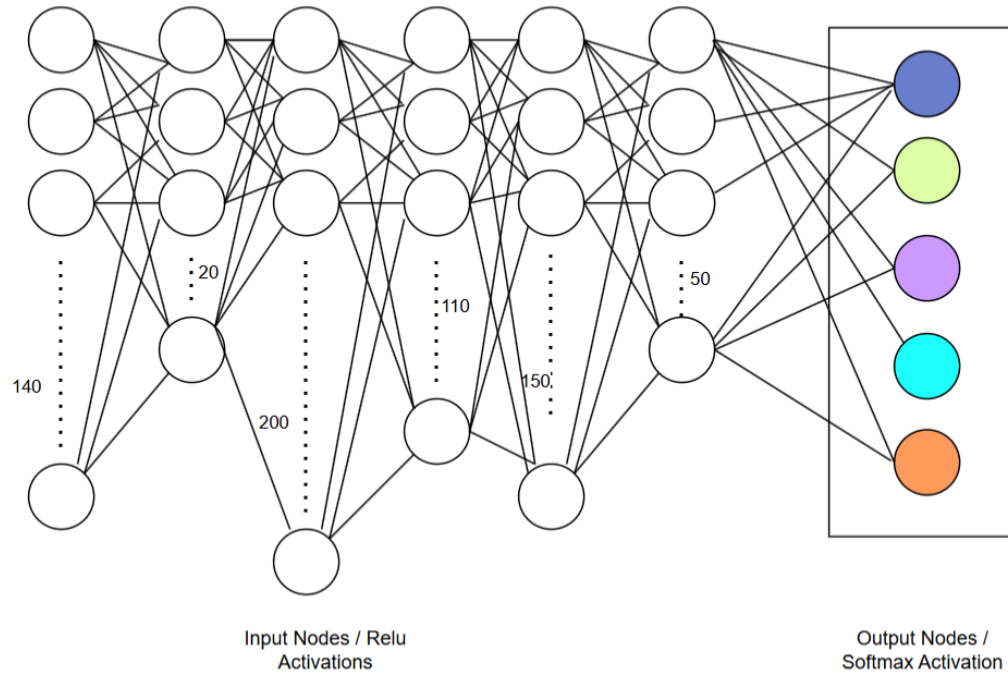


Figure 3.6: Architecture of the Neural Network Model with ReLU and Softmax Activations

3.4 Post-Training Analysis and Visualization

The analysis phase employed a custom analysis framework to validate the model's predictions against unseen test data. Predictions were generated using SoftMax probabilities, mapped to particle class labels for interpretability. This framework seamlessly integrated with training outputs, appending class probabilities and predictions to structured datasets for downstream analysis. Custom performance metrics, such as domain-specific loss functions and accuracy measures, were computed to evaluate the robustness of the model. Additionally, a visualization module was used to generate comprehensive training and diagnostic plots. These included:

- Loss and accuracy trends across epochs to monitor convergence.

- Metrics comparisons between training and validation datasets to detect overfitting or underfitting.
- Gradient flow visualizations to identify optimization bottlenecks.

Gradients are monitored to ensure stability during training. For each layer i , the gradient ∇W_i is analyzed for potential issues like vanishing or exploding gradients. The Gradient Ratio defined as:

$$\text{Gradient Ratio} = \frac{\|\nabla W_i^{(t+1)}\|}{\|\nabla W_i^{(t)}\|} \quad (6)$$

is computed across consecutive training steps. This ratio provides a quantitative measure of the gradient flow through the network. Significant deviations in this ratio—either approaching zero (indicating vanishing gradients) or excessively large (indicating exploding gradients)—trigger diagnostic alerts. Such monitoring enables timely interventions, such as adjusting learning rates, modifying weight initialization schemes, or incorporating additional regularization techniques, ensuring stable and effective training of the neural network.

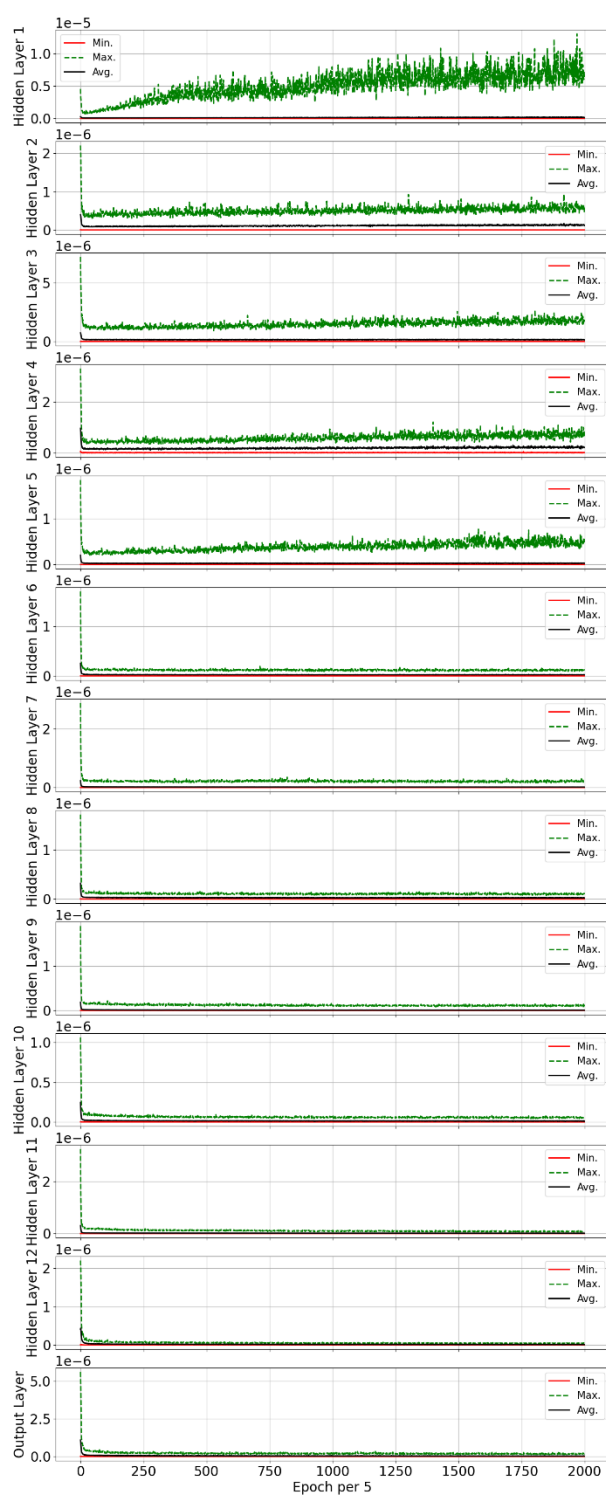


Figure 3.7: Gradient Flow Monitoring Across Model Layers During Training

Figure 3.7 illustrates the gradient flow across all layers of the model during training, highlighting the minimum, maximum, and average gradient magnitudes for each layer over the course of 2000 epochs. This monitoring ensured that issues such as vanishing or exploding gradients were detected early and mitigated, enabling stable and efficient training.

Chapter IV

Experiments

4.1 Dataset

The training and validation datasets were generated through a rigorous Monte Carlo simulation process designed to replicate real-world experimental conditions. First, a "particle gun" event generator emitted particles, such as electrons, muons, pions, kaons, or protons, with randomly assigned momentum and polar angles into the GlueX detector system. To ensure accurate photon tagging, the Bremsstrahlung technique was employed, producing photons tagged between 6 and 11.7 GeV with a resolution of 50 MeV. This setup allowed tracking of charged particles (K^+/K^- , μ^+/μ^- , $p/\bar{p}$, e^-/e^+ , π^+ and π^-) and neutral particles (photons, neutrons) within a polar angle range of 1° to 120° .

Next, the GlueX detector's signal response was simulated using `hdgeant4`, a Geant4-based code, which accurately modeled the physical processes occurring within the detectors. Finally, the simulated signals were smeared according to the resolution of each detector component, ensuring the dataset mirrored the noise and uncertainties inherent in real detector measurements. This comprehensive approach produced high-fidelity datasets, essential for training robust particle classification models.

In this experiment, the dataset is divided into 70% for training and 30% for validation. The validation set is used to tune hyperparameters and evaluate the model during training. A separate, new dataset is used for testing the model, ensuring that the testing data is unseen by the model during both the training and validation phases. This approach helps in obtaining unbiased performance metrics. The dataset is balanced, with

each class having a similar number of samples to prevent class imbalance from affecting model performance.

4.2 Evaluation Metrics

To evaluate the performance of the model, standard classification metrics—precision, recall, F1-score, and support—were computed for each particle class, as well as overall averages (micro, macro, and weighted). Here's a detailed breakdown of how each metric is calculated and its significance in generating the evaluation results:

Precision:

Measures the proportion of true positive predictions out of all positive predictions made by the model. It is defined as:

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} \quad (7)$$

Explanation:

- TP (True Positives): The number of correctly predicted positive instances.
- FP (False Positives): The number of instances incorrectly predicted as positive.
- Interpretation: Precision measures the proportion of correctly predicted positive observations out of all the observations that were predicted as positive. It answers the question: *Of all the instances predicted as positive, how many are correct?*
- Use Case: Precision is crucial when false positives are more critical, e.g., identifying harmful particles where incorrect classification may lead to unnecessary further analysis.

Recall:

Also known as sensitivity or true positive rate, represents the proportion of true positive predictions out of all actual positive instances. It is calculated as:

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative} \quad (8)$$

Explanation:

- FN (False Negatives): The number of instances incorrectly predicted as negative.
- Interpretation: Recall measures the proportion of actual positives that were correctly identified. It answers the question: *Of all the true positive instances, how many were correctly classified?*
- Use Case: Recall is important when missing positive instances (false negatives) is costly, e.g., ensuring no important particle types are missed during classification.

F1-Score:

It is the harmonic mean of precision and recall, providing a single metric that balances both aspects. It is given by:

$$Recall = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (9)$$

Explanation:

- Interpretation: F1-Score is the harmonic mean of Precision and Recall. It balances the tradeoff between Precision and Recall, making it useful when you need to find an optimal balance between these two metrics.
- Use Case: Particularly effective for imbalanced datasets where one class dominates and both false positives and false negatives are significant.

Micro Average:

Micro-averaging computes the metric globally by summing the individual TP, FP, and FN for all classes. It is useful when you want to evaluate performance across all instances equally, regardless of class imbalance. It is calculated as:

$$Precision_{micro} = \frac{\sum TP}{\sum (TP + FP)} \quad (10)$$

Macro Average:

Macro-averaging calculates metrics for each class and then averages them, treating all classes equally. It is effective for evaluating performance on imbalanced datasets by giving equal weight to all classes. It is calculated as:

$$Precision_{macro} = \sum \frac{Precision \text{ per class}}{Number \text{ of classes}} \quad (11)$$

Weighted Average:

Accounts for the support of each class when averaging metrics. It is particularly useful when you need to emphasize performance for classes with more samples.

$$Precision_{weighted} = \frac{\sum (Support \text{ for class} * Precision \text{ for class})}{Total \text{ Support}} \quad (12)$$

4.3 Experiment Settings

The evaluation of the model was conducted in multiple stages, beginning with training on a simulated dataset, followed by validation during training, and ultimately testing on an entirely new simulation dataset to assess generalization performance. The training data consisted of particle types such as electrons, muons, pions, kaons, and protons, each labeled based on their kinematic properties. Throughout training, validation was performed on a separate dataset drawn from the same simulation source to monitor performance metrics and prevent overfitting. Once the model had converged, it was evaluated on a completely new independent simulation dataset, ensuring that it could generalize effectively to unseen particle interactions.

4.3.1 Model Configuration Details

- Input Features: 28, Output Classes: 5
- Hidden Layer Architecture: [140, 20, 200, 110, 150, 50]
- Activation Functions: ReLU (hidden layers), Softmax (output layer)
- Optimizer: Adam
- Learning Rate: 1×10^{-3}
- Batch Size: 2048
- Epochs: 10,000
- Monitoring Intervals: 30 epochs (progress), 5 epochs (evaluation)
- Dropout Rate: 0.1 across all layers
- Gradient Monitoring: Enabled

Additionally, to optimize model performance, Hyperparameter Optimization (HPO) was performed, tuning key parameters such as learning rate, batch size, dropout rate, and the architecture's hidden layer dimensions. The optimized parameters used in the final model are detailed in the following section. This staged evaluation approach, combining training, validation, and testing with distinct datasets, ensured the model's robustness, reliability, and applicability to real-world particle classification tasks.

4.3.2 HPO Configuration Details

The following hyperparameter search space was defined for optimization:

- Number of Trials: 10 (with the potential to extend to 30 for further tuning)
- Neural Network Architecture:
 - Minimum Hidden Layers: 1
 - Maximum Hidden Layers: 10

- Step Size for Layers: 1
 - Minimum Neurons per Layer: 10
 - Maximum Neurons per Layer: 200
 - Step Size for Neurons: 10
- Activation Functions: ReLU (hidden_activations: ['relu'])
- Dropout Rate:
 - Minimum Dropout: 0.0
 - Maximum Dropout: 0.5
 - Dropout Step: 0.05
- Batch Size for Training: 20,000
- Optimizer: Adam (optimizer: 'adam')
- Learning Rate: 1×10^{-3} (learning rate: 1E-3)
- Epochs per Trial: 50,000 (n epochs trial: 50000)
- Monitoring Intervals:
 - Validation monitoring every 30 epochs (read epoch trial: 30)
 - Model performance monitoring every 5 epochs (monitor epoch:5)
- Parallel Coordinates for Analysis: Number of hidden layers, activation functions, and dropout rates were analyzed (parallel coordinates: ['n_hidden_layers', 'activation', 'dropout']).

To ensure efficient training and evaluation of the deep learning model, the experiments were conducted on a high-performance computing server at Jefferson Lab. The system provided robust computational power, leveraging a multi-core processor architecture, extensive memory resources, and optimized CPU scaling for handling large-

scale particle physics datasets. The combination of powerful hardware and a Linux-based environment allowed for seamless execution of Monte Carlo simulations, data preprocessing, and model training. The detailed specifications of the system are as follows:

The system is running on Alma Linux 9.5 (Teal Serval) with a Linux 5.14.0-503.19.1.el9_5.x86_64 kernel, and the architecture is x86-64. It is powered by dual AMD EPYC 9554 64-Core processors, with a total of 256 CPU cores (2 sockets, 64 cores per socket, and 2 threads per core). The CPU frequency range spans from 1500 MHz (minimum) to 3762.99 MHz (maximum), and the CPU boost is enabled with a scaling set at 85% for power-efficient execution. The system has a little-endian byte order and supports a 52-bit physical address and 57-bit virtual address. Memory-wise, the machine is equipped with 1.5 TiB of total RAM, with 120 GiB currently used, 613 GiB free, and 787 GiB in buffered/cache memory. The available memory for computation is 1.4 TiB, and the swap space is 4.0 GiB (2.1 GiB used, 1.9 GiB free).

The hardware is provided by ASUSTeK COMPUTER INC., model RS700A-E12-RS12U, and the firmware version is 1305. The following shows the performance impact for model training.

- The high core count (256 cores) and multi-threading capabilities significantly improved computational parallelism, allowing large-scale Monte Carlo simulations and deep learning training to run efficiently.
- The large memory availability (1.5 TiB) enabled efficient data preprocessing and batch processing for training deep learning models.
- The scalable CPU frequency ensured that computations were dynamically optimized for performance and power efficiency.

- The Linux-based system with high memory bandwidth facilitated smooth execution of Monte Carlo simulations and large-scale data processing required for particle classification.

4.4 Experimental Results

After training the model with the optimized hyperparameters obtained through Hyperparameter Optimization (HPO), the model's performance was evaluated using training and validation datasets. The loss and accuracy trends across epochs were monitored to assess the learning process and ensure effective convergence.

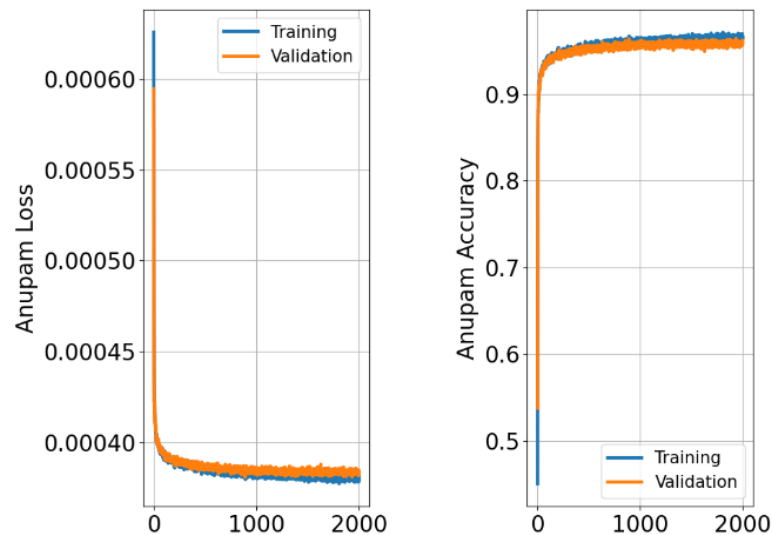


Figure 4.1: Training and Validation Loss and Accuracy Trends for the Model Over 2000 Epochs

The plots in Figure 4.1 illustrate loss and accuracy trends over 2000 epochs:

- Loss: The training and validation loss values showed a consistent decrease, stabilizing towards the end of the training process, indicating that the model converged effectively.

- Accuracy: Both training and validation accuracy demonstrated a steep rise during the initial epochs and gradually stabilized above 90%, highlighting the model's ability to generalize well across datasets.

The performance of the model was evaluated using the confusion matrices for both the validation and testing datasets to assess its ability to classify particle types (K^+/K^- , μ^+/μ^- , $p/\bar{p}$, e^-/e^+ , π^+ and π^-). The validation dataset was derived from the training simulation, while a completely new simulation dataset was utilized for testing to evaluate generalizability.

Table 4.1 Classification Report Highlighting Precision, Recall, and F1-Score of Positive Particle Validation Dataset

<i>Class</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>
K^+	0.63	0.92	0.75
μ^+	0.66	0.95	0.78
p	0.94	0.96	0.95
e^+	0.93	0.99	0.96
<i>Micro Avg</i>	0.77	0.96	0.85
<i>Macro Avg</i>	0.79	0.95	0.86
<i>Weighted Avg</i>	0.79	0.96	0.86

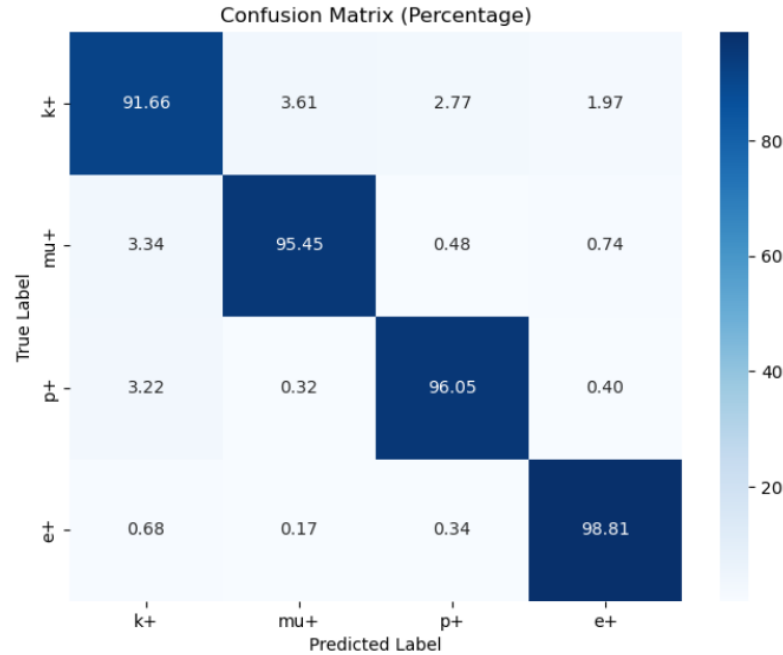


Figure 4.2: Confusion Matrix for Validation Dataset Using Positive Combined-All-Charged-Particles-All-Tkin-26.0 Degree

4.4.1 Validation Performance for Positive Particles

The confusion matrix for the validation dataset (Figure 4.2) highlights the model's strong performance during validation. The model achieved high classification accuracy across different particle types, with K^+ particles correctly classified 91.66% of the time, μ^+ particles with 95.45% accuracy, p particles at 96.05%, and e^+ particles showing the highest accuracy at 98.81%. While the model performed well, some misclassifications were observed, particularly with off-diagonal elements. Notably, 3.34% of K^+ particles were misclassified as μ^+ , and there was minimal confusion for e^+ particles, with only 0.17% misclassified as μ^+ .

Table 4.2 Classification Report Highlighting Precision, Recall, and F1-Score of Positive Particle Test Dataset

<i>Class</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>
K^+	0.88	0.85	0.87
μ^+	0.90	0.98	0.94
p	0.97	0.88	0.92
e^+	0.96	0.98	0.97
<i>Micro Avg</i>	0.93	0.93	0.93
<i>Macro Avg</i>	0.93	0.92	0.92
<i>Weighted Avg</i>	0.93	0.93	0.93

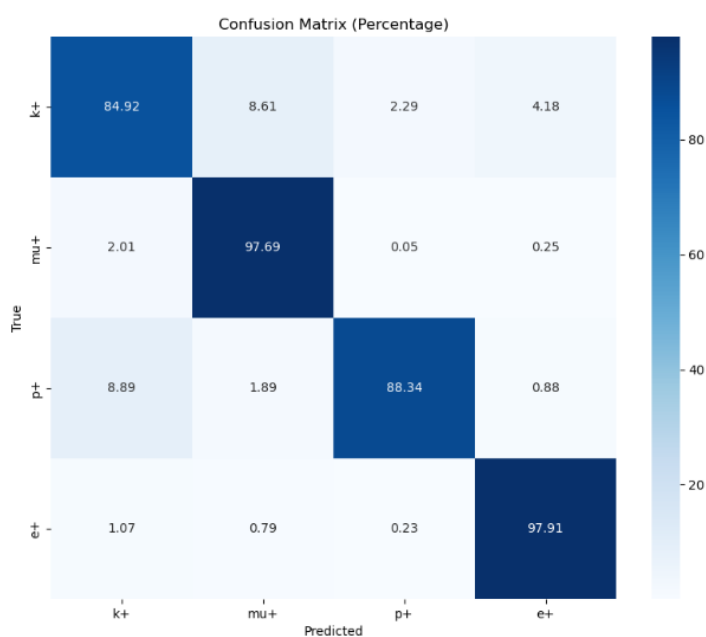


Figure 4.3: Confusion Matrix for Testing Dataset Using Positive Combined-All-Charged-Particles-All-Tkin-26.0 Degree

4.4.2 Testing Performance for Positive Particles

Figure 4.3 and Table 4.2 summarizes the performance of the model for testing dataset for positive particles. The model achieved high classification accuracy across different particle types, with K^+ particles correctly classified 84.92% of the time, μ^+ particles with 97.69% accuracy, p particles at 88.34%, and e^+ particles showing the highest accuracy at 97.91%. While the model performed well, some misclassifications were observed, particularly with off-diagonal elements. Notably, 8.89% of p particles were misclassified as K^+ , and there was minimal confusion for e^+ particles, with only 0.23% misclassified as p.

4.4.3 Insights and Implications

Combining these results demonstrates that the model maintains consistent performance across both validation and testing datasets. The high classification accuracy across both datasets validates the robustness of the training process and hyperparameter optimization. However, areas of misclassification in both validation and testing indicate potential directions for improvement, such as additional training data or fine-tuning the model.

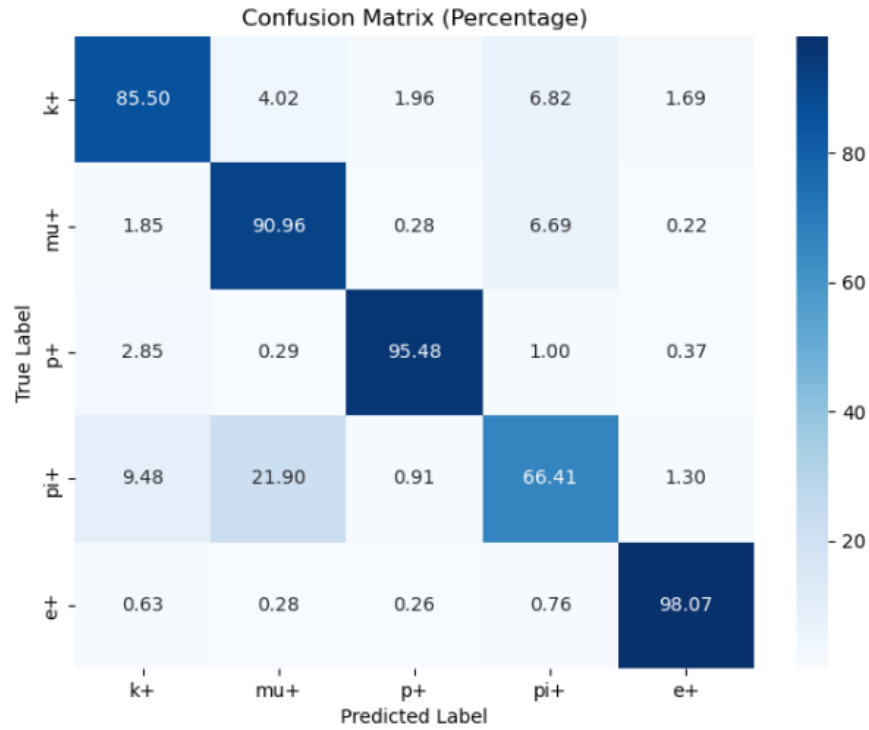


Figure 4.4: Confusion Matrix for Training and Validation with π^+ Particle Class After Applying Data Cuts.

Introducing the π^+ particle class initially reduced classification accuracy, as evident in the confusion matrix (Figure 4.4), with notable misclassification rates such as 21.90% of π^+ particles being identified as μ^+ and 9.48% as K^+ . To address this, momentum-based data cuts were applied to segment the dataset into narrower ranges: 0.25–1 GeV/c, 1–2 GeV/c, 2–3 GeV/c, and 3–5 GeV/c. This refinement enhanced the model's ability to differentiate between particle classes, significantly improving μ^+ classification accuracy while reducing misclassification for other class like k^+ . These adjustments highlight the critical role of data preprocessing in maintaining balanced performance across all particle types.

4.5 Discussion

The results of the trained model indicate strong performance in classifying K^+ , μ^+ , p and e^+ with high accuracy. However, when π^+ (pion) is included in the classification, the overall performance of the model decreases. This suggests that the features distinguishing π^+ (pion) from other particle types may not be sufficiently well-defined within the dataset. To address this, a more detailed feature analysis is required to examine how different particle characteristics contribute to the classification and whether additional feature engineering or selection methods can improve model performance. Additionally, the current approach involves training separate models for each polar angle and then testing them on a dataset of the same angle. While this method ensures that the model learns angle-specific patterns effectively, the next major challenge is to develop a generalized model that can classify particles across all angles without being restricted to a specific angular dataset. This would require training on a combined dataset consisting of multiple angles to ensure that the model learns robust representations independent of angular variations.

An alternative approach using Random Forest was explored, achieving slightly better predictive performance than the neural network in some cases. However, this method required a substantial number of estimators (50–100), resulting in an impractical model size exceeding 20GB. Such computational demands and memory inefficiency make Random Forest less suitable for this application compared to neural networks, which provide a more balanced tradeoff between performance and resource utilization.

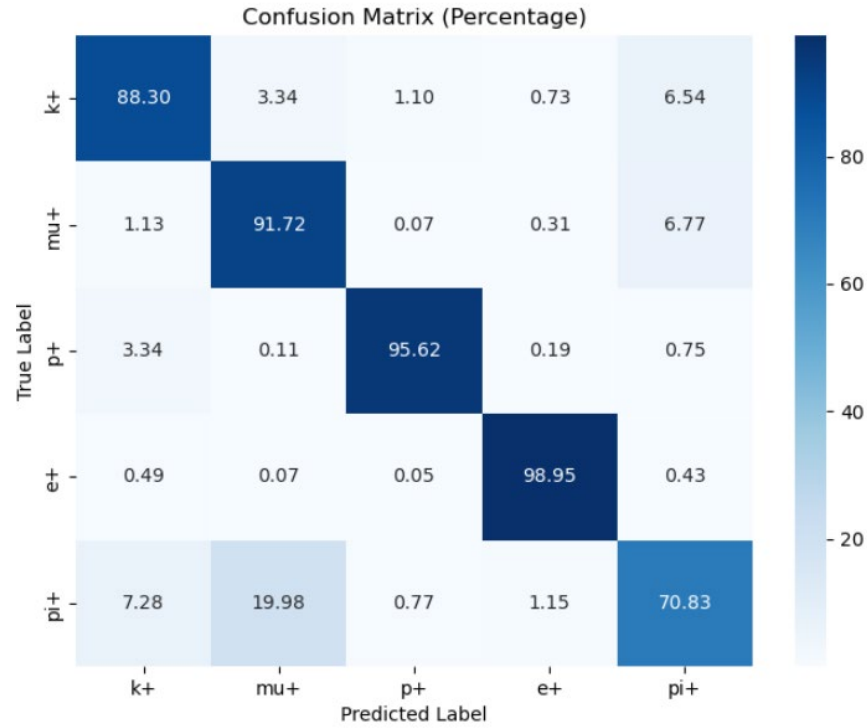


Figure 4.5: Confusion Matrix for Random Forest Model Using the Same Dataset Settings as Neural Networks

Figure 4.5 shows how the inclusion of π^+ decreased the performance of particle identification. Moving forward, the focus will be on expanding the dataset to incorporate multiple angles, refining feature selection, and implementing more advanced techniques such as domain adaptation or angle-invariant feature transformations. These improvements will be crucial in developing a more generalized and robust classification model capable of accurate particle identification across varying experimental conditions.

4.6 Limitations

A limitation of this study is that I have only used the Adam optimizer for training. While Adam is a popular choice and works well in many cases, it might not be the best option for every scenario. Adam combines the benefits of both Momentum and

RMSProp, allowing for adaptive learning rates and faster convergence. However, different optimizers can have their own advantages depending on the model and the problem being solved. For example, pairing Adam with something like Stochastic Gradient Descent (SGD) could help address some of Adam's drawbacks, such as the tendency to get stuck in local minima or overfitting. There are also techniques like learning rate annealing or cyclical learning rates that could potentially make training more stable and improve performance. In future work, it would be useful to explore using a combination of different optimizers to see if they lead to faster convergence, better robustness, or higher performance, providing more insight into how optimization strategies affect the model.

Another limitation of this study is that the model was designed specifically for classifying individual degrees, focusing on a narrow set of angle classifications. While this approach provided useful results for the specific angles considered, it does not address the challenge of creating a general model capable of classifying a broader range of angles. Developing a model that can classify across all possible angles would require more complex architecture or a different approach, potentially involving multi-class classification or an adaptation to handle a wider spectrum of input data. This could be an important direction for future work, exploring the development of more versatile and generalizable models.

Chapter V

Conclusion

This thesis explored the application of machine learning techniques, particularly deep learning, for particle classification using data from the GlueX experiment at Jefferson Lab. A custom neural network architecture was developed and trained using the Adam optimizer and Kaiming initialization, achieving strong performance for particles like K^+/K^- , μ^+/μ^- , $p/\bar{p}$, e^-/e^+ , π^+ and π^- . The use of mini-batches and categorical cross-entropy loss ensured efficient and accurate learning. Despite its success, the inclusion of π^+/π^- in the dataset revealed limitations in the model's ability to maintain consistent performance, suggesting a need for further investigation into feature interactions and dataset characteristics.

Additionally, experiments with Random Forest models demonstrated slightly better accuracy but were hindered by high memory and computational costs, making them impractical for large-scale applications. The study also highlighted the challenges of developing a unified model capable of handling data across all angular ranges without degradation in performance. These findings underscore the need for future research into feature engineering, model optimization, and dataset augmentation to build a more generalizable framework for particle classification. The insights from this work provide a strong foundation for advancing machine learning applications in particle classification.

References

- Agostinelli, J. A. (2003). Geant4—a simulation toolkit. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 250-303.
- Allison, J. A. (2006). Geant4 developments and applications. *IEEE Transactions on Nuclear Science*, 270-278.
- Bergstra, J. &. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 281-305.
- CERN. (n.d.). *ROOT: Data Analysis Framework*. Retrieved from ROOT: analyzing petabytes of data, scientifically: <https://root.cern/>
- Karpathy, A. (2019, April 25). *Andrej Karpathy Blog*. Retrieved from Andrej Karpathy Blog: <https://karpathy.github.io/2019/04/25/recipe/>
- Kasagi, W. D. (2023). Hypernuclear event detection in the nuclear emulsion with Monte Carlo simulation and machine learning. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 168663.
- Kingma, D.P., Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*. International Conference on Learning Representations.
- Palo, W. M. (2024). Neural network applications to improve drift chamber track position measurements. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 169404.

PyTorch Documentation. (n.d.). Retrieved from PyTorch Documentation:

<https://pytorch.org/docs/stable/optim.html>

Radovic, A. W. (2018). Machine learning at the energy and intensity frontiers of particle physics. *Nature*, 560(7716), 41-48.

Sammut, C. &. (2017). *Encyclopedia of Machine Learning and Data Mining*. Springer.

Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 85-117.

Scikit-Learn Documentation. (n.d.). Retrieved from Scikit-Learn Model Evaluation:

https://scikit-learn.org/stable/modules/model_evaluation.html

Wang, X.-Y. D. (2024). Analysis of Strong Coupling Constant with Machine Learning and Its Application. *Chinese Physics Letters*, 031201.

Yang, Z.-X. a.-H.-P. (2024). A neural network approach for orienting heavy-ion collision events. *Physics Letters B*, 138359.

Zhou, K. a.-G. (2024). Exploring QCD matter in extreme conditions with Machine Learning. *Progress in Particle and Nuclear Physics*, 104084.

Zoph, B. &. (2017). *Neural Architecture Search with Reinforcement Learning*. Retrieved from <https://arxiv.org/abs/1611.01578>